

1 Calling Services via PDM Dialogs

1.1 Overview

Purpose

You can call services via dialogs that are sent to the dialog data interface, e.g. from the AIP.



Better call the service directly via the service interface SCS-SIF and not via a PDM dialog. Only call a service via a PDM dialog if the direct call via service interface cannot be used.

Requirements

- Service pack 9. At least Maintenance Pack 555.
- Know-how of the services in use.

Restrictions

- You cannot map list requests (DLG=LIST;n|...) to services.
- You cannot map string commands (DLG=SCMD;n|...) to services.
- Except the return code RET, you cannot pass any results (result parameters, result sets, lists) to the PDM world.

How to proceed

Generate a mapping file for each service that you want to call via a dialog string. The mapping file contains the information which dialog is mapped to which service and any number of conversion rules for the service parameters.

1.2 Tutorial

The tutorial shows how a customer-specific database table is populated with data via a terminal dialog.

In the example you generate a table and a service to create data records in the table. You also create a mapping to a PDM dialog. This way, you can also fill the table with data via an AIP terminal.



This document deals with mapping. As a prerequisite, you need to know how to create tables and how to generate and deploy services.

Customer-specific database table

Use the following SQL statement to create a customer-specific database table. The SQL syntax is "HYDRA-SQL". You must execute the statement with either the utility program `hysql.exe` or `hysql.out` or as a patch.

```
create table u_mapping_example
(
    example_ts datetime,
    example_int integer,
    example_char char(500),
    example_date date,
    example_time integer,
    example_option_yn char(1),
    example_type char(20)
);
revoke all on u_mapping_example from "public";

create unique index "hydadm".u_ix_example_1 on "hydadm".u_mapping_example( example_int );
```

Service

The service `U_CUST_MappingTest.insert` is created as `InterpretedBapiService` in the repository client.

For demonstration purposes, the service contains fields of different data types.

The column `"my_int"` is labeled as unique key in the constraints by `"KEY=1"`.

The column `"is_option"` is defined as type boolean and is stored via the constraints with `"BOOL=Y;N;null;string|"` in the database field `char(1)`.

Services [1]											
Domain...	Domain	Name	Function	Service Type	DLG	List Mode	Description	System Call	Parent	Modified	Is Invalid
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	insert	InterpretedBapiService					Local->U_CUST_MappingTest->Services		

ServiceParameters [7]											
Domain Set	Domain	Service	Acron...	Web Service Type	DB Alias	DB Tabelle	DB Field	Is Special Parameter	Can Equal	Constrai...	Transformation Type
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_int	integer	me	u_mapping_example	example_int	Y	Y	KEY=1	
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	is_option	boolean	me	u_mapping_example	example_option_yn	Y	Y		FCT=BOOLTRANSFORMATION(TRUEINWAL=S FALSEINWAL=W
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_date	datetime	me	u_mapping_example	example_date	Y	Y		
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_string	string	me	u_mapping_example	example_char	Y	Y		
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_time	integer	me	u_mapping_example	example_time	Y	Y		
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_ts	datetime	me	u_mapping_example	example_ts	Y	Y		
Local	U_CUST_MappingTest	U_CUST_MappingTest.insert	my_type	string	me	u_mapping_example	example_type	Y	Y		

...\Interpreter\U_CUST_MappingTest.Configuration.xml for Copy&Paste:

```
<?xml version="1.0"?>
<domains xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <domain name="U_CUST_MappingTest">
    <service name="U_CUST_MappingTest.insert" Function="insert" ServiceType="InterpretedBapiService" listMode="" DLG=""
    SystemCall="">
      <parameter Acronym="is option" ResultSet="" WebServiceType="boolean" DefaultValue="" IsResult="" IsDynamicResult=""
    InputAsArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
    CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
    CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
    TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_option_yn"
    DBAlias="me" DBTabelle="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey=""
    Constraints="BOOL=Y;N;null;string|" />
    </service>
  </domain>
</domains>
```

```
<parameter Acronym="my_date" ResultSet="" WebServiceType="datetime" DefaultValue="" IsResult="" IsDynamicResult=""
InputFromArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_date" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_int" ResultSet="" WebServiceType="integer" DefaultValue="" IsResult="" IsDynamicResult=""
InputFromArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_int" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="KEY=1" />
<parameter Acronym="my_string" ResultSet="" WebServiceType="string" DefaultValue="" IsResult="" IsDynamicResult=""
InputFromArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_char" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_time" ResultSet="" WebServiceType="integer" DefaultValue="" IsResult="" IsDynamicResult=""
InputFromArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_time" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_ts" ResultSet="" WebServiceType="datetime" DefaultValue="" IsResult="" IsDynamicResult=""
InputFromArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_ts" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
<parameter Acronym="my_type" ResultSet="" WebServiceType="string" DefaultValue="" IsResult="" IsDynamicResult=""
InputFromArray="" IsSpecialParameter="Y" IsFilterParameter="" IsMandatory="" CanEqual="Y" CanLike="" CanBetween="" CanIn=""
CanNotEqual="" CanEqualOrNull="" CanLikeOrNull="" CanBetweenOrNull="" CanInOrNull="" CanNotEqualOrNull="" CanGt="" CanLt=""
CanGte="" CanLte="" CanGtOrNull="" CanLtOrNull="" CanGteOrNull="" CanLteOrNull="" HydraAcronym="" HydraResultAcronym=""
TransferEmptyValuesToHydra="" HydraShiftPart="" Reference="" TransformationType="" PlugName="" DBField="example_type" DBAlias="me"
DBTable="u_mapping_example" DBFieldAlternative="" DataObjectName="" ConditionalFieldKey="" Constraints="" />
</service>
</domain>
</domains>
```

Mapping file

The PDM dialogs are sent to the server via the AIP terminal. If you want to redirect the dialogs to the service, you must create a mapping file. The mapping file defines which PDM dialog is "redirected" to which service and which PDM dialog identifications are assigned to which service parameters.



Create the mapping file using the encoding "UTF8 without BOM".

Use the dialog "UMAPTEST.INSERT" to collect data via the AIP.

Name of the mapping file

The mapping file is named after the dialog and is in the JSON format: "UMAPTEST.INSERT.json"

Deployment

You must deploy the mapping file on the server in the JHydradir directory "legacyReplacementMapping"; in the example in the custom scope.

"\\server\\Hydra3\\jhydradir\\MOC\\1\\legacyReplacementMapping\\custom\\UMAPTEST.INSERT.json"

or

"\\server\\mip\\jdir\\MOC\\1\\legacyReplacementMapping\\custom\\UMAPTEST.INSERT.json".

Row no.	Contents
1	{
2	"serviceName": "U_CUST_MappingTest.insert",
3	"dlgName": "UMAPTEST.INSERT",
4	"rules": [
5	{
6	"serviceParam": "my_ts",
7	"ruleType": "timestamp",
8	"dlgParamDate": "DAT",
9	"dlgParamTime": "ZEI"
10	},
11	{
12	"serviceParam": "my_date",
13	"ruleType": "timestamp",
14	"dlgParamDate": "DAT"
15	},
16	{
17	"serviceParam": "my_time",
18	"ruleType": "simple",
19	"dlgParam": "ZEI"
20	},
21	{
22	"serviceParam": "my_int",
23	"ruleType": "simple",
24	"dlgParam": "Z AHL"
25	},
26	{
27	"serviceParam": "is_option",
28	"ruleType": "simple",
29	"dlgParam": "ANAUS"
30	},
31	{
32	"serviceParam": "my_type",
33	"ruleType": "replace_from_to",
34	"dlgParam": "TYP",
35	"fromToMap": {
36	"1": "TYPE_FIRST",
37	"2": "TYPE_SECOND",
38	"3": "TYPE_THIRD"
39	},
40	"defaultValue": "TYPE_UNDEFINED",
41	"onError": "DEFAULT_VALUE"
42	},
43	{
44	"serviceParam": "my_string",
45	"ruleType": "set_constant",
46	"constantValue": "<unspecified>"
47	},
48	{
49	"serviceParam": "my_int",
50	"secondServiceParam": "my_string",
51	"ruleType": "simple_pair",
52	"dlgParam": "VAL_UNIT",
53	"splitter": ";"
54	}
55	]
56	}

Calling the dialog

You can now call the service using a dialog, e.g. via the AIP, or you can simulate the call in the command line of the server.

```
hymw -u1109 -c"DLG=UMAPTEST.INSERT|DAT=05/31/2016|ZEI=86400|ZAHL=-12|ANAU=Y|TYP=2|VAL_UNIT=19;Meter|"
```

Further notes on the mapping file

The first information in the mapping file is: which dialog calls which service (rows 2 and 3).

This information is followed by rules for transferring PDM dialog parameters to service parameters. These rules are processed in the order they are listed in the file.

A rule is only then executed, if the PDM dialog parameter is defined and the service parameter does exist. You can show scenarios like e.g. the service parameter `my_int` is first of all populated by the dialog parameter `ZAHL`. But if the dialog parameter `VAL_UNIT` exists, this dialog parameter overwrites the service parameter `my_int` (rows 21 to 25 and 48 to 54).

Transfer rules

There are different types of rules. The `ruleType` defines the rules:

"ruleType": "simple" (rows 21 to 25)

The `dlgParam` specified is passed to the `serviceParam` specified. Implicit type conversions from the usual forms of presentation are performed. The dot is the decimal separator of decimal numbers.

If the data type of the `serviceParam` is "boolean", the type is converted into "true", if the first character of the `dlgParam` is "1", "J", "j", "Y", "y", "T" or "t" ("T" for "TRUE") (rows 26 to 30).

Service parameters of type "timeStamp" cannot be converted with "ruleType": "simple". For service parameters of type "timeStamp", you must use "ruleType": "timestamp".

"ruleType": "timestamp" (rows 5 to 10)

Service parameters can have the type "timeStamp", which contains a complete time stamp with date and time. Dialog parameters do not know the type timeStamp. Here, date and time are included in two separate dialog parameters.

The specified `serviceParam` is populated with the date from `dlgParamDate` and with the time from `dlgParamTime`.

The date must be in the MM/DD/YYYY format, the time must be in seconds.

TimeStamps containing only a date are equally transferred using the ruleType timeStamp. In this case, only the `dlgParamDate` is used (rows 11 to 15).

The dialog parameters and the service parameters show times in seconds as integers. Times are transferred using the ruleType "simple" (rows 16 to 20).

"ruleType": "replace_from_to"

You use this ruleType to replace values. The dlgParam specified is passed to the serviceParam specified. The value of dlgParam is replaced using the specified table (fromToMap). In the example (rows 31 to 42) the value "1" is replaced with "TYPE_FIRST".

If the text in dlgParam could not be found in the table, the action specified as onError is carried out:

NO_CHANGE

The text in dlgParam is transferred unchanged to serviceParam.

DEFAULT_VALUE

The value specified as defaultValue is transferred to serviceParam.

SKIP_RULE

The rule is skipped, the serviceParam is not changed.

Following these replacement rules, the implicit type conversions are processed as with the rule "simple".

"ruleType": "set_constant"

The serviceParam specified is populated with the constantValue specified (rows 43 to 47).

"ruleType": "simple_pair"

In rare cases, one dialog parameter transfers two separated values. These values must be assigned to two different service parameters. For example: In case of |VAL_UNIT=19;Meter|, the service parameter my_int should be populated with "19" and my_string with "Meter". The rule must define serviceParam, secondServiceParam, dlgParam and splitter.

"ruleType": "user_field_66"

This rule transfers the usual 66 user fields from the dialog parameters to the service parameters. Using this rule, you need not specify 66 individual rules. Accordingly, you enter the placeholder "[n]" as the user field number in the dialog and service parameters. The dialog parameters replace this placeholder with the simple user field number without leading zero. The service parameters replace the placeholder with the corresponding double-digit number, if necessary with leading zero. The placeholder can be at any place in the dialog or service parameter. Each transfer implies the same type conversions as with "ruleType": "simple".

Example:

```
{
  "ruleType": "user_field_66",
  "dlgParam": "CNR.FU.[n]",
```

```
"serviceParam": "batch.userfield.[n] "
},
```

This rule results in the following single transfer rules, for example:

CNR.FU.3	batch.userfield03
CNR.FU.33	batch.userfield33
CNR.FU.44	batch.userfield44

"ruleType": "counter"

This rule is reserved to transfer the standard dialog parameters for "counters' data collection" to service parameters in the future.

"ruleType": "quantities"

This rule is reserved to transfer the standard dialog parameters for "recording of quantities" to service parameters in the future.

Dialog appendices:

In the PDM dialogs, you can add appendices, separated by semicolon, to the actual dialog (identification DLG=...), e.g. DLG=XXX;ASYNC| and DLG=XXX;PLAUS|.

The mapping is performed for the actual dialog only, without appendix. The appendix is virtually transferred to an additional dialog parameter DLG_APPENDIX and can then be respected during parameter conversion. If you need the dialog appendix as service parameter, you must define a transfer rule for this service parameter with dialog parameter DLG_APPENDIX.

Example:

The dialog "DLG=A_UN;ASYNC|ANR=abc|..." is processed as "DLG=A_UN|DLG_APPENDIX=ASYNC|ANR=abc|..." in the mapping.

Error

If the service called detects an error (validation error), the return value for the dialog is -1. The dialog identifications KT and LT provide further information. Especially the language key returned in LT can help to find the error cause.

```
|RET=-1|KT=Validation error occurred|LT=lkInsertAlreadyExists|
```

If a technical runtime error occurs on running the service, the return value for the dialog is again -1. The dialog identifications KT and LT provide further information.

```
|RET=-1|KT=RuntimeException at class: DbUtil at method: getSqlParameter at line:
190: "Unsupported class type for getSqlParameter. Offending type: class
java.lang.Boolean"|LT=1kUnspecifiedError|
```

Use the log file of the JavaServer for a more detailed analysis.

Notes on use

The PDM services (hymw) use a database table (*rwsc_callback_mapping*) to decide which dialogs are redirected to which services. This database table is populated by the Web Service Provider (WSP). The table is not populated immediately after reloading or changing a mapping file.

There are three ways to update this table:

- Automatic: When the Web Service Provider (WSP) is started, the table is automatically updated.
- Service: If you call the service "ServiceDictionary.refresh", the update is started manually.
- Calling the PDM: You can start the update manually by calling the PDM command `DLG=RWSC.MSG|RWSC.ACTION=LEGREPLMAPRENEW|`. You can also execute the command in the command line of the server:

```
hymw -u9999 -c"DLG=RWSC.MSG|RWSC.ACTION=LEGREPLMAPRENEW|"
```